```
import pandas as pd
```

```
df=pd.read_csv('/content/placement.csv')
df
```

	cgpa	resume_score	placed
0	8.14	6.52	1
1	6.17	5.17	0
2	8.27	8.86	1
3	6.88	7.27	1
4	7.52	7.30	1
...	...	...	...
95	6.33	6.38	0
96	8.23	7.76	1
97	6.65	7.78	0
98	8.14	5.63	1
99	6.09	6.61	0

100 rows × 3 columns

Next steps: Generate code with df | New interactive sheet

```
df.head()
```

	cgpa	resume_score	placed
0	8.14	6.52	1
1	6.17	5.17	0
2	8.27	8.86	1
3	6.88	7.27	1
4	7.52	7.30	1

Next steps: Generate code with df | New interactive sheet

```
df.tail()
```

	cgpa	resume_score	placed
95	6.33	6.38	0
96	8.23	7.76	1
97	6.65	7.78	0
98	8.14	5.63	1
99	6.09	6.61	0

```
df.describe()
```

	cgpa	resume_score	placed
count	100.0000	100.000000	100.000000
mean	6.9422	6.930500	0.500000
std	1.1192	0.979608	0.502519
min	5.2700	4.950000	0.000000
25%	5.9800	6.190000	0.000000
50%	6.6200	7.055000	0.500000
75%	8.0450	7.640000	1.000000
max	9.4000	9.060000	1.000000

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 3 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   cgpa          100 non-null    float64
 1   resume_score  100 non-null    float64
 2   placed        100 non-null    int64  
dtypes: float64(2), int64(1)
memory usage: 2.5 KB
```

```
top_left_corner_df=df.iloc[:4,:3]   #iloc[row_range,column_range]
top_left_corner_df
```

	cgpa	resume_score	placed
0	8.14	6.52	1
1	6.17	5.17	0
2	8.27	8.86	1
3	6.88	7.27	1

Next steps: Generate code with top_left_corner_df | New interactive sheet

```
df.loc[1:3,['cgpa','placed']]  #loc[row_range,column_name]
```

	cgpa	placed
1	6.17	0
2	8.27	1
3	6.88	1

```
s=df.axes
s
```

```
[RangeIndex(start=0, stop=100, step=1),
 Index(['cgpa', 'resume_score', 'placed'], dtype='object')]
```

```
p=df.dtypes
p
```

	0
cgpa	float64
resume_score	float64
placed	int64

dtype: object

```
df['cgpa'].dtypes
```

```
dtype('float64')
```

```
b=df.empty #it returns true if there any missing data in dataframe otherwise returns false
b
```

```
False
```

```
df.isnull().sum()
```

	0
cgpa	0
resume_score	0
placed	0

dtype: int64

```
df=df.dropna() #for remove missing row
```

```
i=df.ndim  #num of axes(it is 2)
i
```

```
2
```

```
t=df.shape # shape=n(rows),n(columns)
t
df.shape[0]  #total row in data
df.shape[1]  #total columns in data
```

```
3
```

```
d=df.size #row count*col count
d
```

```
300
```

```
d=df.values  #get anumpy array for df
d
```

Show hidden output

```
df=df.copy()
```

```
p=df.sort_values(by='resume_score')
p
```

	cgpa	resume_score	placed
94	8.01	4.95	1
87	5.62	5.05	0
13	8.50	5.09	1
41	8.68	5.15	1
1	6.17	5.17	0
...	...	...	...
17	7.81	8.71	1
38	6.00	8.72	0
65	6.04	8.75	0
2	8.27	8.86	1
69	8.06	9.06	1

100 rows × 3 columns

Next steps: Generate code with p | New interactive sheet

```
df1=pd.read_csv('/content/Churn_Modelling.csv')
df1.head()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActive
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	

Next steps: Generate code with df1 | New interactive sheet

```
df1.tail()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsAc
9995	9996	15606229	Obijiaku	771	France	Male	39	5	0.00	2	1	
9996	9997	15569892	Johnstone	516	France	Male	35	10	57369.61	1	1	
9997	9998	15584532	Liu	709	France	Female	36	7	0.00	1	0	
9998	9999	15682355	Sabbatini	772	Germany	Male	42	3	75075.31	2	1	
9999	10000	15628319	Walker	792	France	Female	28	4	130142.79	1	1	

```
df1.describe()
```

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActive
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.00000	10000.
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76485.889288	1.530200	0.70550	0.
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62397.405202	0.581654	0.45584	0.
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	0.000000	1.000000	0.00000	0.
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	0.000000	1.000000	0.00000	0.
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97198.540000	1.000000	1.00000	1.
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127644.240000	2.000000	1.00000	1.
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250898.090000	4.000000	1.00000	1.

```
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   RowNumber        10000 non-null  int64
 1   CustomerId       10000 non-null  int64
 2   Surname          10000 non-null  object
 3   CreditScore      10000 non-null  int64
 4   Geography        10000 non-null  object
 5   Gender           10000 non-null  object
 6   Age              10000 non-null  int64
 7   Tenure           10000 non-null  int64
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64
 10  HasCrCard        10000 non-null  int64
 11  IsActiveMember   10000 non-null  int64
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

```
top_left_corner_df=df1.iloc[:4,:3]   #iloc[row_range,column_range]
top_left_corner_df
```

	RowNumber	CustomerId	Surname
0	1	15634602	Hargrave
1	2	15647311	Hill
2	3	15619304	Onio
3	4	15701354	Boni

Next steps: Generate code with `top_left_corner_df` New interactive sheet

```
df1.loc[1:3,['CustomerId','Surname']]
```

	CustomerId	Surname
1	15647311	Hill
2	15619304	Onio
3	15701354	Boni

```
s=df.axes
s
```

```
[RangeIndex(start=0, stop=100, step=1),
 Index(['cgpa', 'resume_score', 'placed'], dtype='object')]
```

```
p=df1.dtypes
p
```

	0
RowNumber	int64
CustomerId	int64
Surname	object
CreditScore	int64
Geography	object
Gender	object
Age	int64
Tenure	int64
Balance	float64
NumOfProducts	int64
HasCrCard	int64
IsActiveMember	int64
EstimatedSalary	float64
Exited	int64

dtype: object

```
df1['Balance'].dtypes
```

```
dtype('float64')
```

```
b=df1.empty
b
```

```
False
```

```
df1.isnull().sum()
```

	0
RowNumber	0
CustomerId	0
Surname	0
CreditScore	0
Geography	0
Gender	0
Age	0
Tenure	0
Balance	0
NumOfProducts	0
HasCrCard	0
IsActiveMember	0
EstimatedSalary	0
Exited	0

dtype: int64

```
df1=df1.dropna()
```

```
i=df1.ndim
i
```

```
2
```

```
t=df.shape
t
```

```
(100, 3)
```

```
df.shape[0]  #total row in data
df.shape[1]
```

```
3
```

```
d=df.size
d
```

```
300
```

```
d=df.values
d
```

Show hidden output

```
p1=df1.sort_values(by='Age')
p1
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCar
1678	1679	15569178	Kharlamov	570	France	Female	18	4	82767.42	1	
4556	4557	15796231	Nwankwo	681	France	Female	18	1	98894.39	1	
9520	9521	15673180	Onyekaozulu	727	Germany	Female	18	2	93816.70	2	
9526	9527	15665521	Chiazagomekpele	642	Germany	Male	18	5	111183.53	2	
9572	9573	15641688	Collier	644	Spain	Male	18	7	0.00	1	
...	...	...	...	...	...	...	...	...	...	...	...
3531	3532	15653251	Hickey	408	France	Female	84	8	87873.39	1	
3033	3034	15578006	Yao	787	France	Female	85	10	0.00	2	
2458	2459	15813303	Rearick	513	Spain	Male	88	10	0.00	2	
6443	6444	15764927	Rogova	753	France	Male	92	3	121513.31	1	
6759	6760	15660878	T'ien	705	France	Male	92	1	126076.24	2	

10000 rows × 14 columns

Next steps: Generate code with p1 New interactive sheet

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```